

Learning Objectives

A systematic study of this chapter will help the reader understand the following concepts:

- Creating a list
- Accessing and manipulating list elements
- Merging lists
- Converting lists to vectors

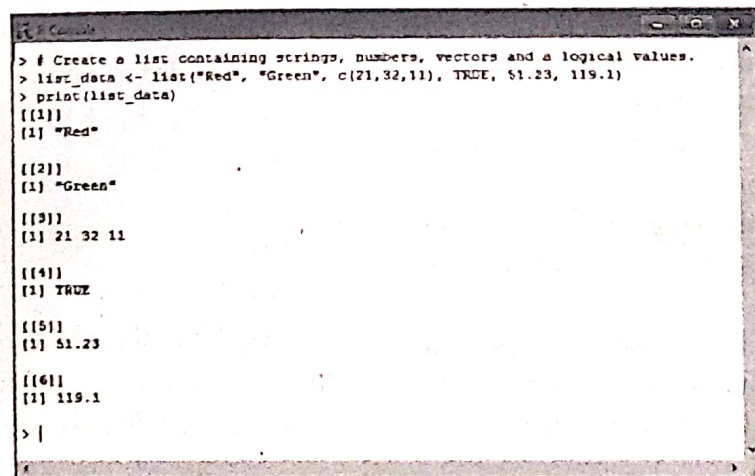
3.1 INTRODUCTION

A list in R constitutes of different objects such as strings, numbers, and vectors. Further, it can include another list within it. Lists in R can also include matrices and functions making it functionally richer than other existing list utilities. Lists can be obtained as return values from many modelling functions such as `t.test()` for t tests, `lm()` for liner models, etc. Customized lists can be created to store useful information such as grocery list, shopping list, list of books, and so on. Most of these lists are simple with nominal and ordinal values, but in many cases there is a need for storing complex information such as the configuration details of a phone; in such cases, the list function of R can be used.

3.2 CREATING A LIST

A list can be created using the `list()` function, which takes in different R objects and stores the values in the database.

Figure 3.1 demonstrates the creation of an anonymous list `list_data`, using the `list()` function, containing strings, numbers, vectors, and logical values as data objects. After creating the list it can be viewed with the help of the `print` statement.



```
> # Create a list containing strings, numbers, vectors and a logical values.
> list_data <- list("Red", "Green", c(21,32,11), TRUE, 51.23, 119.1)
> print(list_data)
[[1]]
[1] "Red"

[[2]]
[1] "Green"

[[3]]
[1] 21 32 11

[[4]]
[1] TRUE

[[5]]
[1] 51.23

[[6]]
[1] 119.1

> |
```

Fig. 3.1 Creation of lists

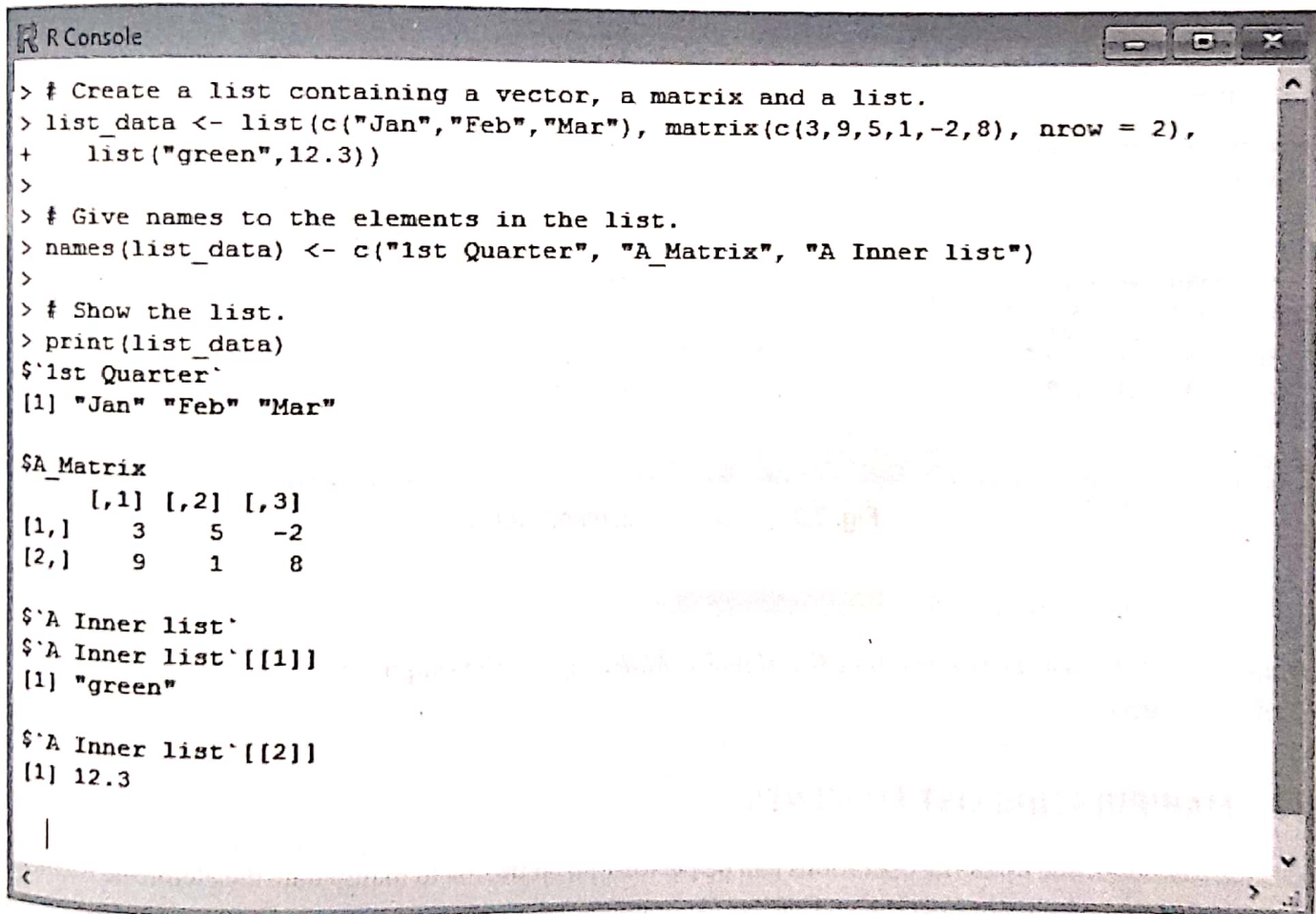
3.2.1 Creating a Named List

Lists in R have a special property wherein names can be assigned to the previously defined set of elements in the list. These forms of named lists provide correct information about a particular category of data with many different attributes. Named lists can be created using the *names()* function.

Example 3.1 Write the command in R console to create a list containing a vector, a matrix, and a list. Also give names to the elements in the list and display the list.

Solution:

Figure 3.2 creates a named list using the *names()* function. First a list called *list_data* containing a vector and a matrix and another list called *list* are created. Later using the *names()* function, a vector with the names to these lists is created. The created list is viewed using the *print* statement.



```

R Console
> # Create a list containing a vector, a matrix and a list.
> list_data <- list(c("Jan", "Feb", "Mar"), matrix(c(3, 9, 5, 1, -2, 8), nrow = 2),
+   list("green", 12.3))
>
> # Give names to the elements in the list.
> names(list_data) <- c("1st Quarter", "A_Matrix", "A Inner list")
>
> # Show the list.
> print(list_data)
$`1st Quarter`
[1] "Jan" "Feb" "Mar"

$A_Matrix
  [,1] [,2] [,3]
[1,]   3   5  -2
[2,]   9   1   8

$`A Inner list`
$`A Inner list`[[1]]
[1] "green"

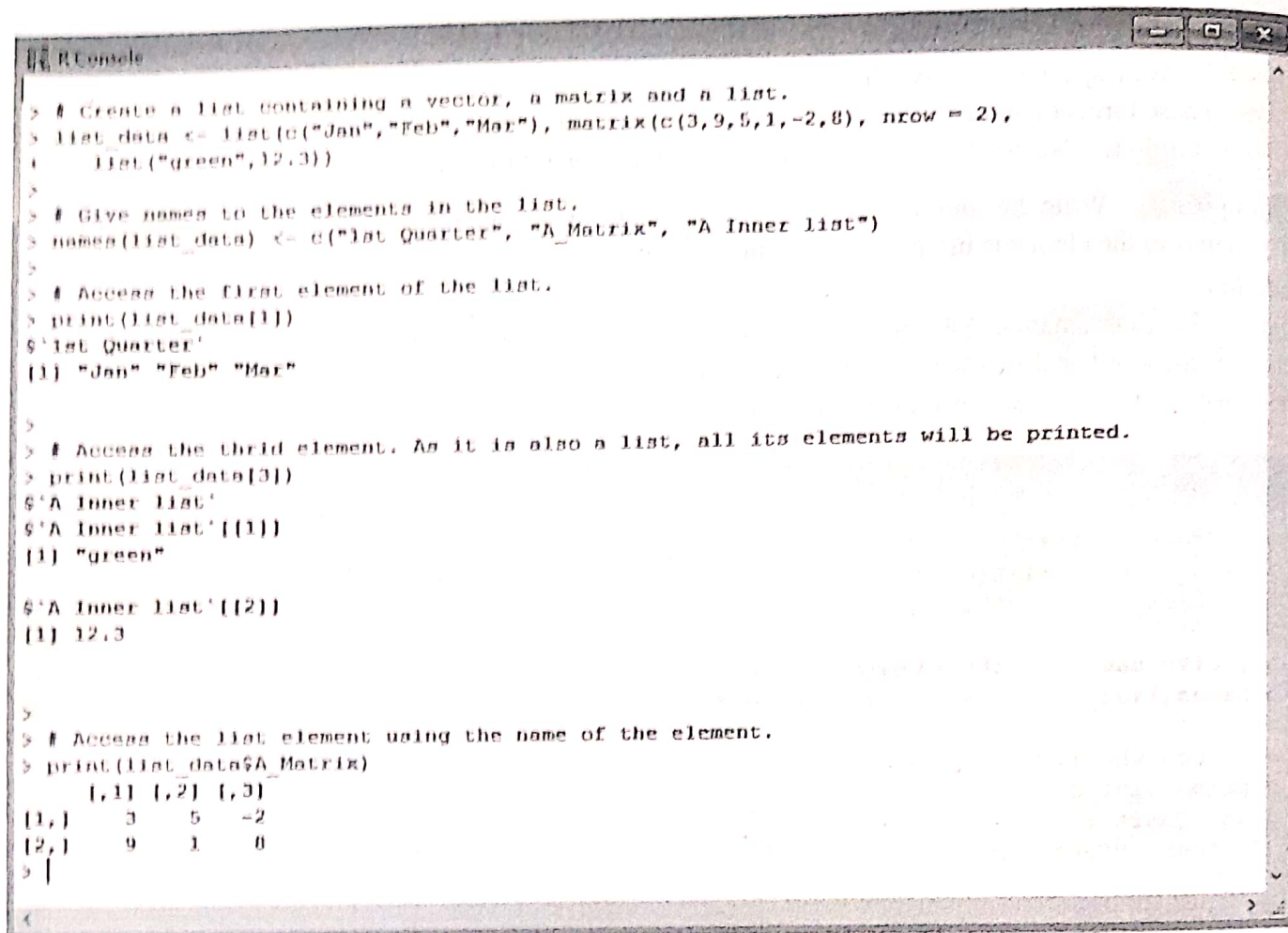
$`A Inner list`[[2]]
[1] 12.3
  
```

Fig. 3.2 Creating a named list

3.3 ACCESSING LIST ELEMENTS

The elements present in a normal list can be accessed using the index value of that element. For a named list, the elements are accessed based on their names.

In Fig. 3.3, a list called *list_data* is created using the *list()* function and later the list elements are given names using the *names()* function. So to access the first element from *list_data*, the index value of the list is accessed; here the list elements are indexed starting from 1 and not from 0. Therefore printing *list_data[1]* gives the names of the first and later elements in the list. If the third element in the list is considered, which is another list containing two different elements, printing *list_data[3]* gives the list name and the two elements. The elements in the list can also be accessed with their names using the '\$' sign appended with the name of



```

> # Create a list containing a vector, a matrix and a list.
> list_data <- list(c("Jan", "Feb", "Mar"), matrix(c(3,9,5,1,-2,8), nrow = 2),
+   list("green", 12.3))
>
> # Give names to the elements in the list.
> names(list_data) <- c("1st Quarter", "A_Matrix", "A Inner list")
>
> # Access the first element of the list.
> print(list_data[1])
$'1st Quarter'
[1] "Jan" "Feb" "Mar"

>
> # Access the third element. As it is also a list, all its elements will be printed.
> print(list_data[3])
$'A Inner list'
$'A Inner list'[[1]]
[1] "green"

$'A Inner list'[[2]]
[1] 12.3

>
> # Access the list element using the name of the element.
> print(list_data$A_Matrix)
      [,1] [,2] [,3]
[1,]    3    5   -2
[2,]    9    1    8
>

```

Fig. 3.3 Accessing elements from a list

the element in the list. Hence printing `list_data$A_Matrix` gives the output of the third element of the list which is a matrix.

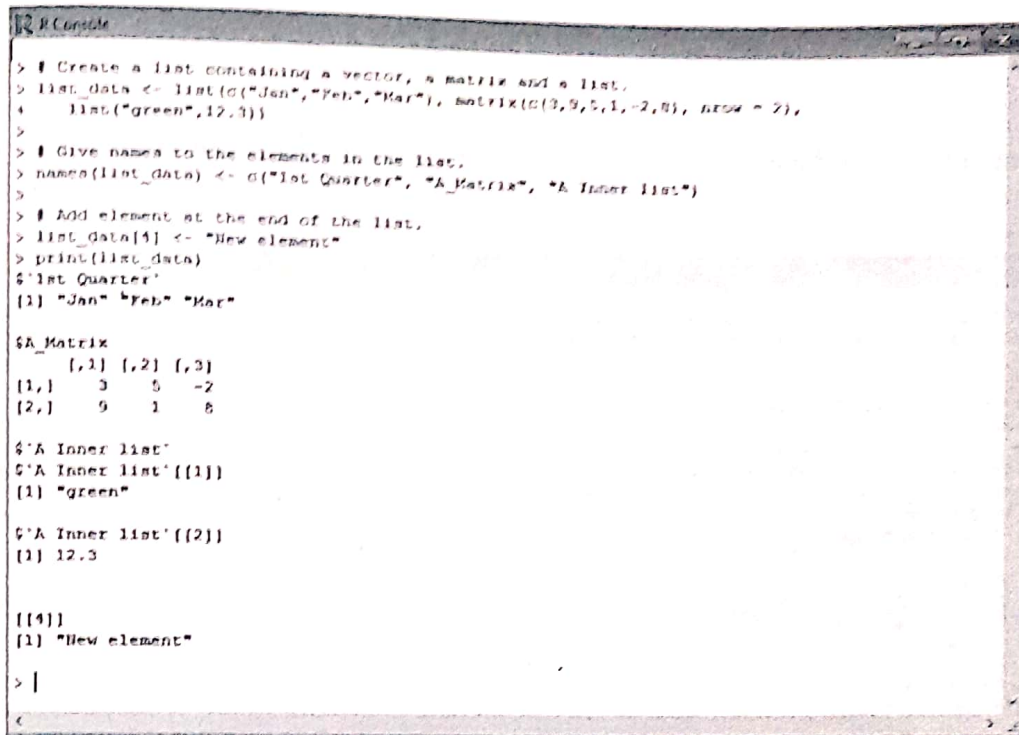
3.4 MANIPULATING LIST ELEMENTS

Addition, deletion, and updating operations can be performed on the list to manipulate the elements. Elements can be added and deleted only at the end of the list, whereas, the update operation can be performed on any element in the list irrespective of its position.

Example 3.2 Write the command in R console to add a new element at the end of the list and display the same.

Solution:

Figure 3.4 depicts the addition of an element to a list. Consider the example list created in Fig. 3.2, which consists of only three elements. To add the fourth element to this list, append the element (which is a string here) "New element" to the 4th indexed position of the list, that is, `list_data[4]`. To view the added element in the list, print the list and the four elements are displayed.



```

> # Create a list containing a vector, a matrix and a list.
> list_data <- list(c("Jan", "Feb", "Mar"), matrix(c(3, 9, 5, 1, -2, 8), nrow = 2),
+   list("green", 12.3))
>
> # Give names to the elements in the list.
> names(list_data) <- c("1st Quarter", "A_Matrix", "A Inner list")
>
> # Add element at the end of the list.
> list_data[4] <- "New element"
> print(list_data)
$'1st Quarter'
[1] "Jan" "Feb" "Mar"

$A_Matrix
  [,1] [,2] [,3]
[1,]  3   5  -2
[2,]  9   1   8

$'A Inner list'
$'A Inner list'[[1]]
[1] "green"

$'A Inner list'[[2]]
[1] 12.3

[[4]]
[1] "New element"

> |

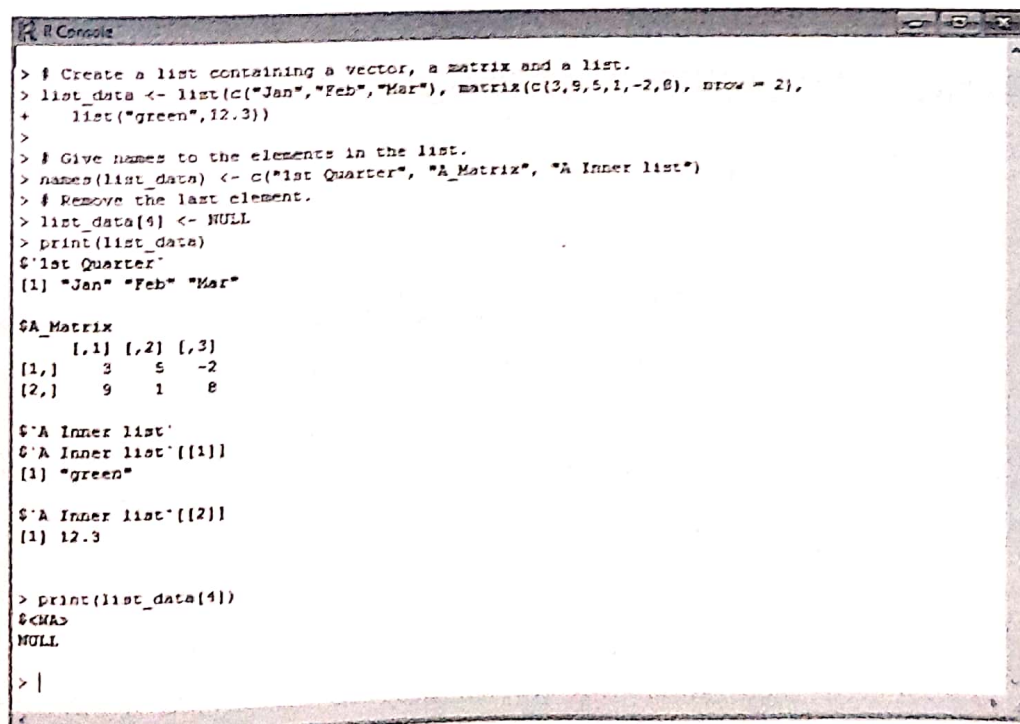
```

Fig. 3.4 Adding an element to the list

Example 3.3 Write the command in R console to delete the fourth element from a list and display the resultant list.

Solution:

Here, in Fig. 3.5, the fourth element, "New element" can be deleted by setting its value to NULL. Printing *list_data* shows only three elements in the list and when the fourth element is tried to be printed, it shows NULL.



```

> # Create a list containing a vector, a matrix and a list.
> list_data <- list(c("Jan", "Feb", "Mar"), matrix(c(3, 9, 5, 1, -2, 8), nrow = 2),
+   list("green", 12.3))
>
> # Give names to the elements in the list.
> names(list_data) <- c("1st Quarter", "A_Matrix", "A Inner list")
>
> # Remove the last element.
> list_data[4] <- NULL
> print(list_data)
$'1st Quarter'
[1] "Jan" "Feb" "Mar"

$A_Matrix
  [,1] [,2] [,3]
[1,]  3   5  -2
[2,]  9   1   8

$'A Inner list'
$'A Inner list'[[1]]
[1] "green"

$'A Inner list'[[2]]
[1] 12.3

> print(list_data[4])
$NULL
NULL

> |

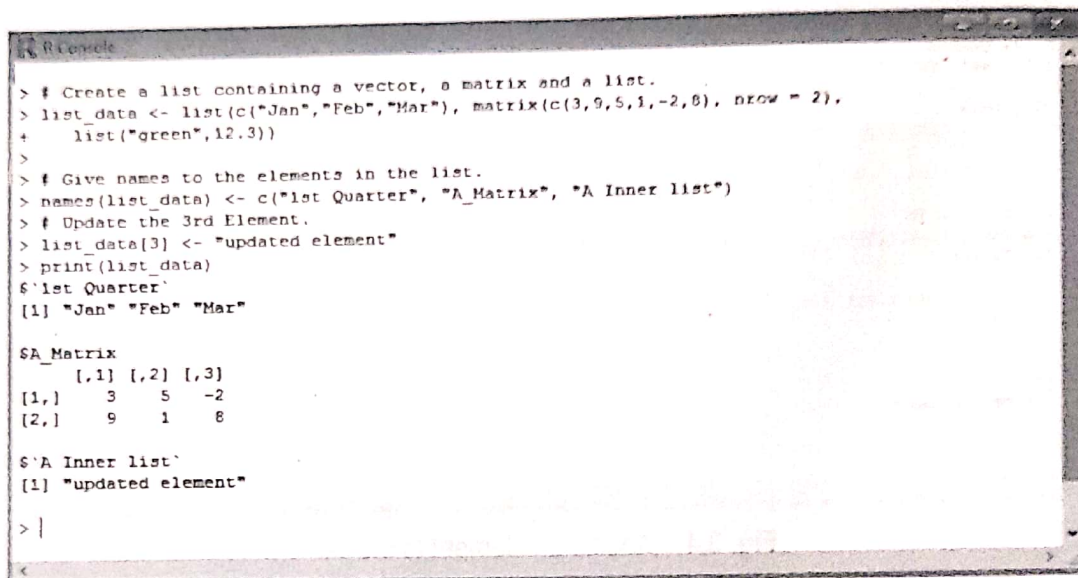
```

Fig. 3.5 Deleting an element from a list

Example 3.4 Write the command in R console to update the third element of the list and display the resultant list.

Solution:

In Fig. 3.6, we update the third element of *list_data* as "updated element". This is then checked by printing the list.



```

> # Create a list containing a vector, a matrix and a list.
> list_data <- list(c("Jan", "Feb", "Mar"), matrix(c(3,9,5,1,-2,8), nrow = 2),
+   list("green", 12.3))
>
> # Give names to the elements in the list.
> names(list_data) <- c("1st Quarter", "A_Matrix", "A Inner list")
> # Update the 3rd Element.
> list_data[3] <- "updated element"
> print(list_data)
$1st Quarter
[1] "Jan" "Feb" "Mar"

$A_Matrix
      [,1] [,2] [,3]
[1,]    3    5  -2
[2,]    9    1    8

$A Inner list
[1] "updated element"

>

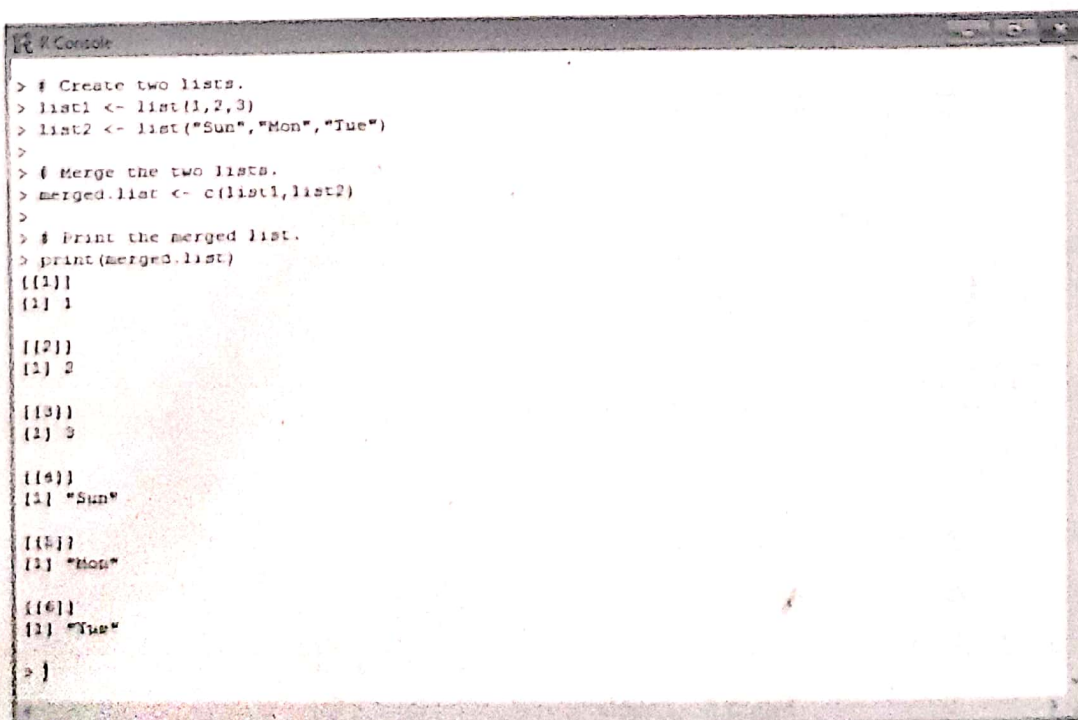
```

Fig. 3.6 Updating a list element

3.5 MERGING LISTS

Multiple lists can be merged by placing all those lists in a single *list()* function.

Figure 3.7 shows two lists *list1* and *list2* created using the *list()* function. Now placing them in a single *list()* function gives a merged list called *merged.list*, which contains the elements of both the lists placed sequentially.



```

> # Create two lists.
> list1 <- list(1,2,3)
> list2 <- list("Sun", "Mon", "Tue")
>
> # Merge the two lists.
> merged.list <- c(list1, list2)
>
> # Print the merged list.
> print(merged.list)
[[1]]
[1] 1

[[2]]
[1] 2

[[3]]
[1] 3

[[4]]
[1] "Sun"

[[5]]
[1] "Mon"

[[6]]
[1] "Tue"

>

```

Fig. 3.7 Merging two lists

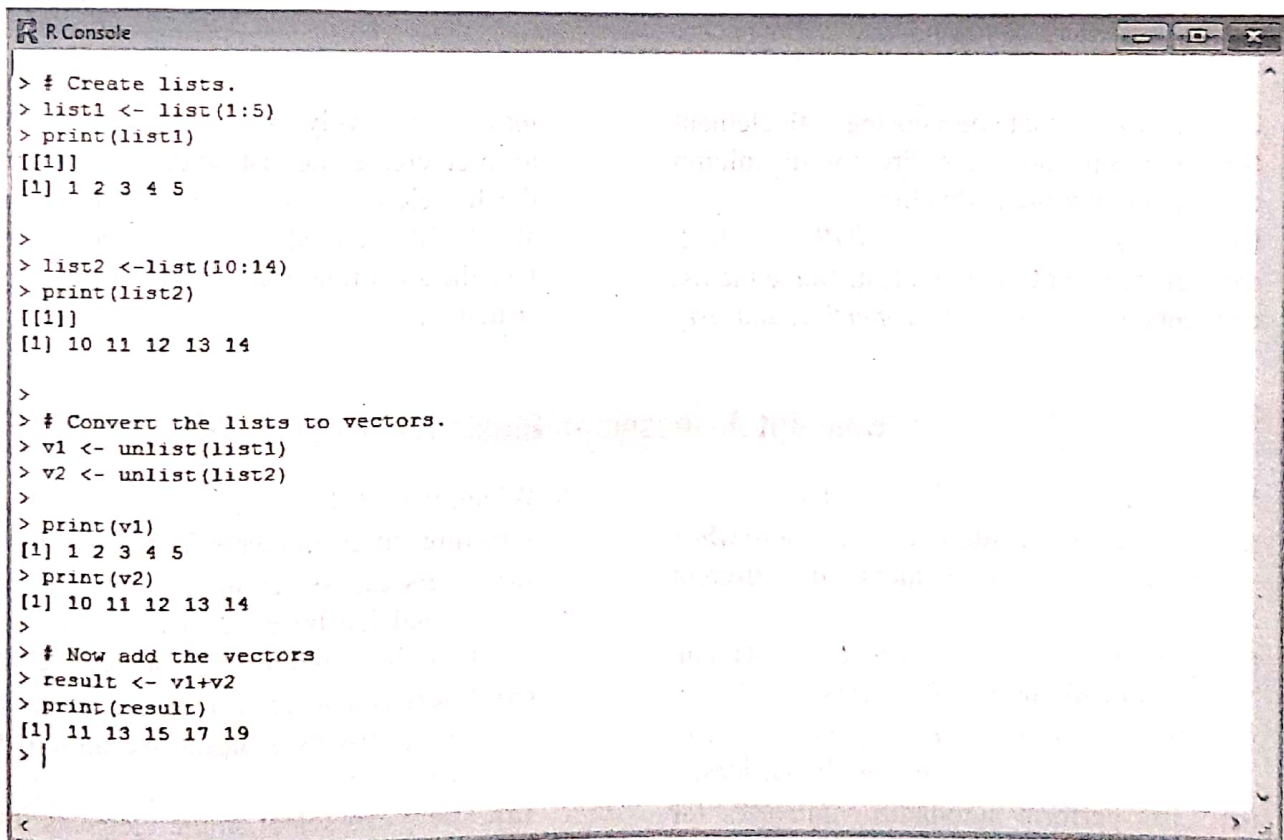
3.6 CONVERTING LISTS TO VECTORS

There are some special cases where lists are converted to vectors for further manipulation of the data elements. This is done because in addition to the basic operations of add, delete, and update, there are other arithmetic operations that can be easily applied when the operands are in vector form. For this conversion, a special function called `unlist()` is used; this function takes the input as list and produces vectors as depicted in detail in Example 3.5.

Example 3.5 Write the command in R console to create two lists, each containing 5 elements. Convert the lists into vectors and perform addition on the two vectors. Display the resultant vector.

Solution:

As shown in Fig. 3.8, two lists namely, *list1* and *list2* are initially created. Later with the help of the `unlist()` function, both the lists are converted to vectors named *v1* and *v2*. Addition, subtraction, and multiplication of two vectors can be performed, which was not possible if the data was in the form of a list.



```

> # Create lists.
> list1 <- list(1:5)
> print(list1)
[[1]]
[1] 1 2 3 4 5

>
> list2 <- list(10:14)
> print(list2)
[[1]]
[1] 10 11 12 13 14

>
> # Convert the lists to vectors.
> v1 <- unlist(list1)
> v2 <- unlist(list2)
>
> print(v1)
[1] 1 2 3 4 5
> print(v2)
[1] 10 11 12 13 14
>
> # Now add the vectors
> result <- v1+v2
> print(result)
[1] 11 13 15 17 19
>
  
```

Fig. 3.8 Converting lists to vectors

Programming Questions

- 3.1 Perform the following operations using lists:
 - (a) Construct a list, named *my_list* that contains the variables *my_vector*, *my_matrix* and *my_factor* as list components.
 - (b) Construct a list, named *my_super_list*, which now contains the four predefined variables listed.
 - (c) Print the structure of *my_super_list* with the `str()` function.
 - (d) Change the code that built *my_list* by adding names to the components. Use the names *mat*, *vec*, and *fac*, respectively, for *my_matrix*, *my_vector*, *my_factor*.
 - (e) Print the list to the console and inspect the output.

- 3.2 Being a huge movie fan, a user decides to start storing information on good movies with the help of lists. Create the variable *shining_list*. The list contains the movie title as *title*, then the actor names as *actors*, and finally the review scores factor as *reviews*. Pay attention to naming the objects correctly.

```

Console - / 
> # Create actors and reviews
> actors_vector <- c("Jack Nicholson", "Shelley Duvall", "Danny Lloyd", "Scatman Crothers",
"Barry Nelson")
> reviews_factor <- factor(c("Good", "OK", "Good", "Perfect", "Bad", "Perfect", "Good"),
+                           ordered = TRUE, levels = c("Bad", "OK", "Good", "Perfect"))
> |

```

- 3.3 Create a list *lst* that contains the fifth element from the top and the entire fourth column of *prop*. Do not name this list.
- 3.4 Create a new list, *skills*, which contains *top*, *cont*, *prop*, and *lst*. Name the list elements *topics*, *context*, *properties*, and *list_*

info, respectively. Users can either choose to first create the list with *list()* and name the list elements later using *names()* or can also build a named list in a single command. Use the *str()* function on *skills* to display the structure.

Concept Assessment Questions

- 3.1 What is the advantage of using a list?
- Lists perform automatic coercion when confronted with elements of different types.
 - Unlike vectors and matrices, lists can contain all kinds of R objects.
 - Performing arithmetic operations with lists is easier compare to with matrices.
 - Lists perform automatic summaries for every row and column.
- 3.2 Which function should users use to display the structure of a list in R?
- 3.3 Which of the following data objects can be stored in a list?
- Vectors
 - Matrices
 - Lists
- 3.4 Which of the following symbols allow users to extract a single element from an unnamed list.
- `[]`
 - `[[ ]]`
 - `$`
 - `()`

- 3.5 Which two of the following statements about selecting single elements from a list are true?
- Users can select single elements from a named list by making use of subsetting by indices and double square brackets.
 - Users can select single elements from a named list by making use of subsetting by logicals.
 - Users can select single elements from a named list by making use of subsetting by name and single squared brackets.
 - Users can select single elements from a named list by making use of subsetting by the dollar sign.
- 3.6 Which two notations should users use to add a vector to a list?
- `[]`
 - `[[ ]]`
 - `$`
 - `()`
 - `{ }`
- 3.7 What would the following code print?
- ```

> x <- list(1, "a", TRUE, 1 + 4i)
x

```